

ACM输出输出模板 + 练习

(C++,Java,Python,go,JS)

1. 多行输入，每行两个整数

C++

Java

Python

Go

JavaScript (Node)

2. 多组数据，每组第一行为n, 之后输入n行两个整数

C++

Java

Python

Go

JS

3. 若干行输入，每行输入两个整数，遇到特定条件终止

C++

Java

Python

Go

JavaScript

4. 若干行输入，遇到0终止，每行第一个数为N，表示本行后面有N个数

C++

Java

Python

Go

JavaScript(Node)

5. 若干行输入，每行包括两个整数a和b，由空格分隔，每行输出后接一个空行。

C++

Java

Python

Go

Js

6. 多组n行数据，每行先输入一个整数N，然后在同一行内输入M个整数,每组输出之间输出一个空行。

C++

Java

Python

Go

JavaScript

7. 多组测试样例，每组输入数据为字符串，字符用空格分隔,输出为小数点后两位

C++

Java

Python

Go

JavaScript(Node)

8. 多组测试用例，第一行为正整数n, 第二行为n个正整数，n=0时，结束输入，每组输出结果的下面都输出一...

C++

Java

Python

Go

Js

9. 多组测试数据，每组数据只有一个整数，对于每组输入数据，输出一行，每组数据下方有一个空行。

C++

Java

Python

Go

JS

10. 多组测试数据，每个测试实例包括2个整数M，K（ $2 \leq k \leq M \leq 1000$ ）。M=0，K=0代表输入结束。

C++

Java

Python

JS

Go

11. 多组测试数据，首先输入一个整数N，接下来N行每行输入两个整数a和b, 读取输入数据到Map

C++

Java

Python

Js

Go

12. 多组测试数据。每组输入一个整数n，输出特定的数字图形

C++

Java

Python

Go

JS

13. 多行输入，每行输入为一个字符和一个整数，遇到特殊字符结束

C++

Java

Python

Go

Js

14. 第一行是一个整数n，表示一共有n组测试数据, 之后输入n行字符串

C++

Java

Python

Go

Js

15. 第一行是一个整数n，然后是n组数据，每组数据2行，每行为一个字符串，为每组数据输出一个字符串， ...

C++

Java

Python

Go

JS

16. 多组测试数据，第一行是一个整数n，接下来是n组字符串，输出字符串

C++

Java

Python

Go

Js

17. 多组测试数据，每组测试数据的第一行为整数N ($1 \leq N \leq 100$)，当N=0时，输入结束，第二行为N个正...

C++

Java

Python

Go

JS

18. 一组输入数据，第一行为n+1个整数，逆序插入n个整数，第二行为一个整数m, 接下来有m行字符串，并...

C++

Java

Python

Go

Javascript

19. 多组测试数据，每行为n+1个数字，输出链表或对应的字符串

C++

Java

python

Go

Js

20. 多组输入，每组输入包含两个字符串，输出字符串

C++

Java

Python

Go

JS

21. 一组多行数据，第一行为数字n, 表示后面有n行，后面每行为1个字符加2个整数，输出树节点的后序遍历...

C++

Java

Go

Python

JS

22. 多组测试数据，首先给出正整数N，接着输入两行字符串，字符串长度为N

C++

Java

Python

Go

JS

23. 多组测试数据。每组输入占一行，为两个字符串，由若干个空格分隔

C++

Java

Go

Python

Js

24. 多组测试数据，每组第一行为两个正整数n和m，接下来m行，每行3个整数, 最后一行两个整数

C++

Java

Python

Go

JS

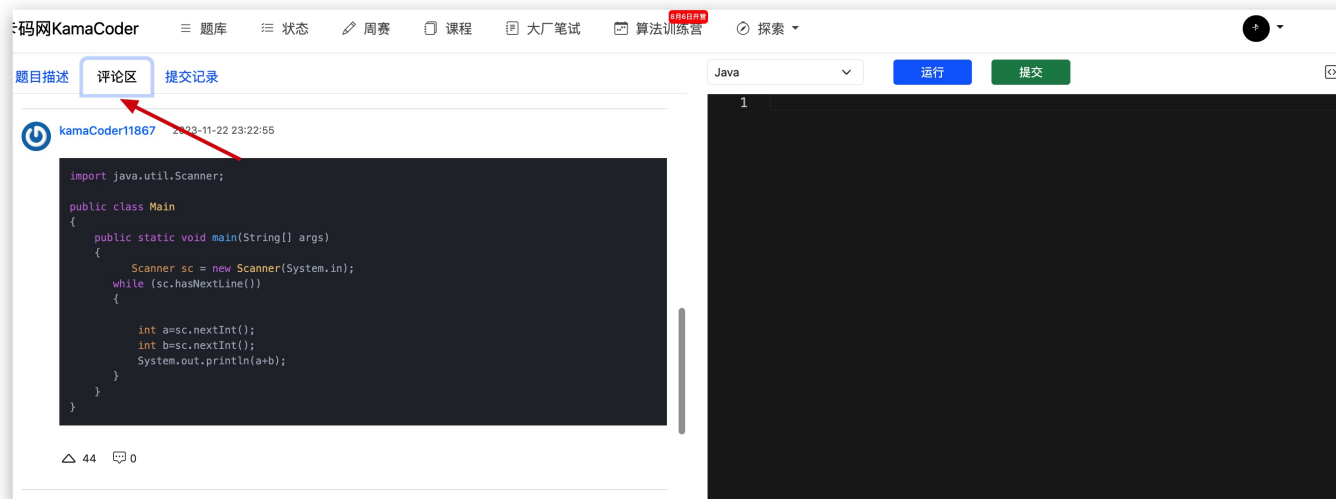
很多录友苦于不熟悉 ACM 输入输出结构，在笔试和面试的时候，处理数据输入输出就花费了大量的时间，以至于算法题没写完，甚至是 根本就写不对输入输出的方式。

下面，我针对常见的25种 ACM输入与输出方式，给大家总结了模板写法，包括了C++、Java、Python、Go、JS等主流编程语言。

大家可以拿去直接“背诵”。

每种输入输出，都配套的对应的[卡码网](#)练习题，每道题目我都给出练习题的链接。

注意本PDF只给出每种情况的ACM输入输出的模板写法，没有题目的完整代码，想看题目完整代码可以看[卡码网](#)题目的「评论区」



如果还没有掌握编程语言，可以报名卡码网的[编程语言课](#)

[ACM输出输出模版PDF下载](#)

1. 多行输入，每行两个整数

练习题：[A+B问题](#)

C++

```
1  #include<iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      while (cin >> a >> b) cout << a + b << endl;
6  }
```

Java

```

1  import java.lang.*;
2  import java.util.*;
3
4  public class Main{
5      public static void main(String[] args){
6          Scanner in = new Scanner(System.in);
7          while(in.hasNextInt()){
8              int a = in.nextInt();
9              int b = in.nextInt();
10             System.out.println(a+b);
11         }
12     }
13 }

```

Python

```

1  import sys
2  # 接收输入
3  for line in sys.stdin:
4      a, b = line.split(' ')
5      # 输出
6      print(int(a) + int(b))
7      # 输出换行
8      print()

```

Go

```

1  package main
2
3  import "fmt"
4
5  func main(){
6      var a, b int
7      for {
8          _, err := fmt.Scanf("%d %d",&a, &b)
9          if err != nil {
10             break
11         }
12         fmt.Println(a + b)
13     }
14 }

```

JavaScript (Node)

```
1 // 引入readline模块来读取标准输入
2 const readline = require('readline');
3
4 // 创建readline接口
5 const rl = readline.createInterface({
6   input: process.stdin,
7   output: process.stdout
8 });
9
10 // 处理输入和输出
11 function processInput() {
12   rl.on('line', (input) => {
13     // 将输入按空格分割成a和b的数组
14     const [a, b] = input.split(' ').map(Number);
15
16     // 计算a和b的和并输出
17     const sum = a + b;
18     console.log(sum);
19   });
20 }
21
22 // 开始处理输入
23 processInput();
```

2. 多组数据，每组第一行为n，之后输入n行两个整数

练习题：[A+B问题II](#)

C++

```
1 #include<iostream>
2 using namespace std;
3 int main() {
4   int n, a, b;
5   while (cin >> n) {
6     while (n-- > 0) {
7       cin >> a >> b;
8       cout << a + b << endl;
9     }
10  }
11 }
```


Java

```
1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          while (scanner.hasNext()) {
7              int n = scanner.nextInt();
8              while (n-- > 0) {
9                  int a = scanner.nextInt();
10                 int b = scanner.nextInt();
11                 System.out.println(a + b);
12             }
13         }
14     }
15 }
```

Python

```
1  while 1:
2      try:
3          N = int(input())
4          for i in range(N):
5              l = list(map(int, input().split()))
6              print(sum(l))
7      except:
8          break
```

Go

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      var n, a, b int
7      for {
8          _, err := fmt.Scan(&n)
9          if err != nil {
10              break
11          }
12          for n > 0 {
13              _, err := fmt.Scan(&a, &b)
14              if err != nil {
15                  break
16              }
17              fmt.Println(a + b)
18              n--
19          }
20      }
21  }
```

JS

```

1  // 引入readline模块来读取标准输入
2  const readline = require("readline");
3
4  // 创建readline接口
5  const rl = readline.createInterface({
6      input: process.stdin,
7      output: process.stdout,
8  });
9
10 function processInput() {
11     let pathArr = [];
12     rl.on("line", (input) => {
13         let path = input.split(" ").map(Number);
14         // 将输入转为数组，根据length属性判断输入数字的个数
15         if (path.length == 1) {
16             pathArr.push(path);
17         } else {
18             const [a, b] = path
19             const sum = a + b;
20             console.log(sum);
21         }
22     });
23
24 }
25 processInput();

```

3. 若干行输入，每行输入两个整数，遇到特定条件终止

练习题：A+B问题III

C++

```

1  #include<iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      while (cin >> a >> b) {
6          if (a == 0 && b == 0) break;
7          cout << a + b << endl;
8      }
9  }

```

Java

```
1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          while (scanner.hasNext()) {
7              int a = scanner.nextInt();
8              int b = scanner.nextInt();
9              if (a == 0 && b == 0) {
10                 break;
11             }
12             System.out.println(a + b);
13         }
14     }
15 }
16
```

Python

```
1  import sys
2
3  while True:
4      s = input().split() # 一行一行读取
5      a, b = int(s[0]), int(s[1])
6      if not a or not b: # 遇到 0, 0 则中断
7          break
8      print(a + b)
```

Go

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var a, b int
7      for {
8          _, err := fmt.Scan(&a, &b)
9          if err != nil {
10              break
11          }
12          if a == 0 && b == 0 {
13              break
14          }
15          fmt.Println(a + b)
16      }
17  }

```

JavaScript

```

1  // 引入readline模块来读取标准输入
2  const readline = require('readline');
3
4  // 创建readline接口
5  const rl = readline.createInterface({
6      input: process.stdin,
7      output: process.stdout
8  });
9
10 function preoceeInput() {
11     rl.on('line', (input) => {
12         const [a, b] = input.split(' ').map(Number);
13         // # 遇到 0, 0 则中断
14         if (a === 0 && b === 0) {
15             return;
16         } else {
17             console.log(a + b);
18         }
19     });
20 }
21
22 preoceeInput()

```

```

1  // 使用 Node.js 的 readline 模块来模拟 C++ 中的 cin 和 cout 操作
2  function main() {
3      // 导入readline模块
4      const readline = require('readline');
5
6      // 创建readline接口
7      const rl = readline.createInterface({
8          input: process.stdin, // 从标准输入读取数据
9          output: process.stdout // 将输出写入标准输出
10     });
11
12     // 监听用户的输入事件
13     rl.on('line', (input) => {
14         // 将输入拆分为两个数，并将其转换为数字
15         const [a, b] = input.split(' ').map(Number);
16
17         // 判断输入的两个数是否都为0
18         if (a === 0 && b === 0) {
19             rl.close(); // 如果是，则关闭输入流，结束程序
20         } else {
21             console.log(a + b); // 否则，计算并输出两数之和
22         }
23     });
24 }
25
26 main(); // 调用主函数开始程序的执行

```

4. 若干行输入，遇到0终止，每行第一个数为N，表示本行后面有N个数

练习题：4. A + B问题IV

C++

```

1  #include<iostream>
2  using namespace std;
3  int main(){
4      int n, a;
5      while (cin >> n) {
6          if (n == 0) break;
7          // 计算累加值
8          int sum = 0;
9          while (n--> 0) {
10             cin >> a;
11             sum += a;
12         }
13         cout << sum << endl;
14     }
15 }

```

Java

```

1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          while (scanner.hasNext()) {
7              int n = scanner.nextInt();
8              if (n == 0) {
9                  break;
10             }
11             int sum = 0;
12             for (int i = 0; i < n; i++) {
13                 sum += scanner.nextInt();
14             }
15             System.out.println(sum);
16         }
17     }
18 }
19 }

```

Python

```

1  import sys
2
3  for line in sys.stdin:
4      nums = line.split()
5      nums = list(map(int, nums))
6      n = nums[0]
7      if not n:
8          break
9      print( sum(nums[-n:]) )

```

Go

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var n, a int
7      for {
8          _, err := fmt.Scan(&n)
9          if err != nil {
10             break
11          }
12          if n == 0 {
13             break
14          }
15          sum := 0
16          for n > 0 {
17              _, err := fmt.Scan(&a)
18              if err != nil {
19                 break
20             }
21             sum += a
22             n--
23          }
24          fmt.Println(sum)
25      }
26  }

```

JavaScript(Node)


```

1  // 引入readline模块来读取标准输入
2  const readline = require('readline');
3
4  // 创建readline接口
5  const rl = readline.createInterface({
6      input: process.stdin,
7      output: process.stdout
8  });
9
10 function preoceeInput() {
11     rl.on('line', (input) => {
12         // 读入每行数据，将其转换为数组
13         const line = input.split(' ').map(Number);
14         // 判断读入的第一个数字是否为0
15         if (line[0] === 0) {
16             return;
17         } else {
18             let sum = 0;
19             for (let i = 1; i < line[0] + 1; i++) {
20                 sum += line[i];
21             }
22             console.log(sum);
23         }
24     });
25 }
26
27 preoceeInput()

```

5. 若干行输入，每行包括两个整数a和b，由空格分隔，每行输出后接一个空行。

练习题：A+B问题 VII

C++

```

1  #include<iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      while (cin >> a >> b) cout << a + b << endl << endl;
6  }

```

Java

```
1  import java.util.Scanner;
2  public class Main{
3      public static void main(String[] args){
4          Scanner sc = new Scanner(System.in);
5          while(sc.hasNextLine()){
6              int a = sc.nextInt();
7              int b = sc.nextInt();
8              System.out.println(a + b);
9              System.out.println();
10         }
11     }
12 }
```

Python

```
1  while True:
2      try:
3          x, y = map(int, (input().split()))
4          print(x + y)
5          print()
6      except:
7          break
```

Go

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      var a, b int
7      for {
8          _, err := fmt.Scan(&a, &b)
9          if err != nil {
10              break
11          }
12          fmt.Printf("%d\n\n", a+b)
13      }
14  }
15
```

Js

```
1  const readline=require('readline');
2  const rl=readline.createInterface({
3      input:process.stdin,
4      output:process.stdout
5  });
6  function Sum(){
7      rl.on("line",(input)=>{
8          const [a,b]=input.split(' ').map(Number);
9          console.log(a+b+"\n");
10     });
11 }
12 Sum();
```

6. 多组n行数据，每行先输入一个整数N，然后在同一行内输入M个整数,每组输出之间输出一个空行。

练习题：A+B问题 VIII

C++

```
1  #include<iostream>
2  using namespace std;
3  int main() {
4      int n, m, a;
5      // 输入多组数据
6      while (cin >> n) {
7          // 每组数据有n行
8          while(n-->0) {
9              cin >> m;
10             int sum = 0;
11             // 每行有m个
12             while(m-->0) {
13                 cin >> a;
14                 sum += a;
15             }
16             cout << sum << endl;
17             cout << endl;
18         }
19     }
20 }
```

Java

```
1  import java.util.Scanner;
2  public class Main{
3      public static void main(String[] args){
4          Scanner sc = new Scanner(System.in);
5          while(sc.hasNextLine()){
6              int N = sc.nextInt();
7              // 每组有n行数据
8              while(N-- > 0){
9                  int M = sc.nextInt();
10                 int sum = 0;
11                 // 每行有m个数据
12                 while(M-- > 0){
13                     sum += sc.nextInt();
14                 }
15                 System.out.println(sum);
16                 if(N > 0) System.out.println();
17             }
18         }
19     }
20 }
21 }
```

Python

```
1  while 1:
2      try:
3          N = int(input())
4          for i in range(N):
5              n = list(map(int, input().split()))
6              if n[0] == 0:
7                  print()
8                  continue
9              print(sum(n[1:]))
10             if i<N-1:
11                 print()
12         except:
13             break
```

Go

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var N, M, num, sum int
7
8      // 无限循环，直到没有更多的输入数据
9      for {
10         // 尝试读取一组测试数据的数量
11         if _, err := fmt.Scan(&N); err != nil {
12             break // 如果读取失败（例如，输入结束），则退出循环
13         }
14
15         // 处理每一组测试数据
16         for i := 0; i < N; i++ {
17             sum = 0 // 初始化当前组的数字和为0
18             fmt.Scan(&M) // 读取当前组的数字数量
19
20             // 读取并累加当前组的所有数字
21             for j := 0; j < M; j++ {
22                 fmt.Scan(&num) // 读取一个数字
23                 sum += num      // 将该数字加到总和中
24             }
25
26             // 输出当前组的数字和
27             fmt.Println(sum)
28
29             // 如果不是当前组的最后一个数字，输出一个空行
30             if i < N-1 {
31                 fmt.Println()
32             }
33         }
34     }
35 }

```

JavaScript

```

1  // 引入readline模块来读取标准输入
2  const readline = require('readline');
3
4  // 创建readline接口
5  const rl = readline.createInterface({
6      input: process.stdin,
7      output: process.stdout
8  });
9
10 function preoceeInput() {
11     let n;
12     rl.on('line', (input) => {
13         // 读入每行数据，将其转换为数组
14         const readline = input.split(' ').map(Number);
15         if (readline.length === 1) {
16             n = readline[0];
17         } else {
18             let sum = 0;
19             for (let i = 1; i < readline[0] + 1; i++) {
20                 sum += readline[i];
21             }
22             if (n > 1) {
23                 console.log(sum + "\n");
24                 n--;
25             } else { // 如果是第n行 只输出结果不换行
26                 console.log(sum);
27             }
28         }
29     });
30 }
31
32 preoceeInput()

```

7. 多组测试样例，每组输入数据为字符串，字符用空格分隔,输出为小数点后两位

练习题：7. 平均绩点

C++

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      string s;
5      while (getline(cin, s)) { // 接受一整行字符串
6          for(int i = 0; i < s.size(); i++) { // 遍历字符串
7
8              }
9          }
10 }

```

```

1  #include <iostream>
2  #include <stdio.h>
3  int main() {
4      float sum = 10.0;
5      int count = 4;
6      printf("%.2f\n", sum / count);
7  }

```

Java

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          while (in.hasNextLine()) {
7              String line = in.nextLine(); // 接收一整行字符串作为输入
8              String[] items = line.split(" "); // 字符串分割成数组
9              for (String item : items) { // 遍历数组
10
11                  }
12              }
13          }
14
15
16  }

```

```

1 public class Main {
2     public static void main(String[] args) {
3         double avg = 3.25;
4         System.out.printf("%.2f\n", avg);
5     }
6 }

```

Python

```

1 while 1:
2     try:
3         n = input().replace(" ", "").replace("A", "4").replace("B", "3").r
4         eplace("C", "2").replace("D", "1").replace(
5             "F", "0")
6         s = 0
7         for i in n:
8             if i not in '43210':
9                 print('Unknown')
10                break
11            s += int(i)
12        else:
13            print(f"{s / len(n):.2f}")
14    except:
15        break

```

Go


```

1  package main
2
3  import(
4      "bufio"
5      "fmt"
6      "os"
7      "strings"
8  )
9
10 func main(){
11     //创建一个bufio.Reader对象，用于从标准输入（即键盘）读取数据
12     reader := bufio.NewReader(os.Stdin)
13     for{
14         s, _, err := reader.ReadLine()
15         s_list := strings.Split(string(s), " ")
16         //如果err的值不等于nil，则表示输入结束
17         if err != nil{
18             break
19         }
20         for i := 0; i < len(s_list); i++){
21
22         }
23     }
24 }

```

```

1  fmt.Println(fmt.Sprintf("%.2f", 3.14159))

```

JavaScript(Node)

```

1  // 引入readline模块读取输入
2  const readline = require("readline");
3  // 创建readline接口
4  const rl = readline.createInterface({
5      input: process.stdin,
6      output: process.stdout,
7  });
8
9  function processInput() {
10
11      rl.on("line", (input) => {
12          let arr = input.split(" ");
13          // 遍历
14          for (let i = 0; i < arr.length; i++) {
15
16              }
17          });
18      }
19      processInput();
20

```

```

1  let svg = 3.14159
2  console.log(svg.toFixed(2))

```

8. 多组测试用例，第一行为正整数n，第二行为n个正整数，n=0时，结束输入，每组输出结果的下面都输出一个空行

练习题 8. 摆平积木

C++

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main() {
6      int n;
7      while (cin >> n) {
8          if (n == 0) break;
9          // 创建vector
10         vector<int> nums = vector<int>(n, 0);
11         // 输入一组数据
12         for (int i = 0; i < n; i++) {
13             cin >> nums[i];
14         }
15         // 遍历
16         for (int i = 0; i < n; i++) {
17             cout << nums[i];
18         }
19         cout << result << endl;
20         cout<< endl;
21     }
22 }

```

Java

```

1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class Main {
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          while (scanner.hasNext()) {
8              Integer size = scanner.nextInt();
9              if (size == 0) {
10                 break;
11             }
12             // 创建list
13             ArrayList<Integer> list = new ArrayList<>();
14             // 添加一组数据到list中
15             for (int i = 0; i < size; i++) {
16                 int num = scanner.nextInt();
17                 list.add(num);
18             }
19             // 遍历
20             for (int i = 0; i < list.size(); i++) {
21                 System.out.println(list.get(i));
22             }
23             System.out.println(res);
24             System.out.println();
25         }
26     }
27 }

```

Python

```

1  while 1:
2      try:
3          n = int(input())
4          if n == 0:
5              break
6          ls = list(map(int, input().split()))
7          # 遍历
8          for i in ls:
9              #操作
10                 print(moves//2)
11             print()
12     except:
13         break

```

Go

```
1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var n int
9      for {
10         _, err := fmt.Scanf("%d", &n)
11         if err != nil || n == 0 {
12             break
13         }
14         nums := make([]int, n)
15
16         for i := 0; i < n; i++ {
17             // 读取一个整数，存放在数组中
18             fmt.Scanf("%d", &nums[i])
19         }
20         for i := 0; i < n; i++ {
21             fmt.Println(nums[i])
22         }
23         fmt.Println(result)
24         fmt.Println()
25     }
26 }
```

Js

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6
7      while ((line = await readline())) {
8          // 读取输入数组长度
9          let size = parseInt(line)
10         if(size == 0) break
11         let input = await readline()
12         // 将输入的数字转换成整数数组
13         let arr = input.split(" ").map(Number);
14         // 遍历数组
15         for (let i = 0; i < size; i++) {
16             // arr[i]
17         }
18         console.log(res);
19         console.log();
20     }
21 })();

```

9. 多组测试数据，每组数据只有一个整数，对于每组输入数据，输出一行，每组数据下方有一个空行。

练习题：9. 奇怪的信

C++

```

1  #include<iostream>
2  using namespace std;
3  int main() {
4      int n, a;
5      while (cin >> n) {
6          while (n != 0) {
7              a = (n % 10); // 获取各位数据
8              n = n / 10;
9          }
10         cout << result << endl;
11         cout << endl;
12     }
13 }

```

Java

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          while (in.hasNextInt()) {
7              int n = in.nextInt();
8              while (n > 0) {
9                  int tmp = n % 10; // 获取各位数据
10                 n /= 10;
11             }
12             System.out.println(res);
13             System.out.println();
14         }
15     }
16 }
```

Python

```
1  while 1:
2      try:
3          n=input()
4          s=0
5          for i in n:
6              print(int(i))
7          print(s)
8          print()
9      except:
10         break
```

Go

```

1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var n, a int
9      for {
10         _, err := fmt.Scanf("%d", &n)
11         if err != nil || n == 0 {
12             break
13         }
14         for n != 0 {
15             a = n % 10
16             n = n / 10
17         }
18         fmt.Println(result)
19         fmt.Println()
20     }
21 }

```

JS

```

1  const readline = require("readline");
2  const r1 = readline.createInterface({
3      input: process.stdin,
4      output: process.stdout,
5  });
6
7  const iter = r1[Symbol.asyncIterator]();
8
9  const read_line = async () => (await iter.next()).value;
10
11  let line = null;
12
13  (async function () {
14      while ((line = await read_line())) {
15          const arr = line.split("").map((item) => Number(item));
16          for (let i = 0; i < arr.length; i++) {
17              }
18              console.log(sum, "\n");
19          }
20      })();

```


10. 多组测试数据，每个测试实例包括2个整数M，K ($2 \leq k \leq M \leq 1000$)。M=0，K=0代表输入结束。

练习题 10. 运营商活动

C++

```
1  #include<iostream>
2  using namespace std;
3  int main() {
4      int m, k;
5      while (cin >> m >> k) {
6          if (m == 0 && k == 0) break;
7          int sum = m + m / k; // 第一轮回得到总话费
8
9          cout << sum << endl;
10     }
11 }
```

Java

```
1  import java.util.*;
2
3  public class Main{
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          while (sc.hasNextInt()) {
7              int m = sc.nextInt();
8              int k = sc.nextInt();
9              if (m == 0 && k == 0) break;
10             int sum = 0;
11
12             System.out.println(sum);
13         }
14     }
15 }
```

Python

```

1  while True:
2      M, K = map(int, input().split())
3      if M == 0 and K == 0:
4          break
5      res = M
6      print(res)

```

JS

```

1  const readline = require("readline");
2  const r1 = readline.createInterface({
3      input: process.stdin,
4      output: process.stdout,
5  });
6
7  const iter = r1[Symbol.asyncIterator]();
8
9  const read_line = async () => (await iter.next()).value;
10
11  let line = null;
12
13  (async function () {
14      while ((line = await read_line())) {
15          const [m, k] = line.split("").map((item) => Number(item));
16          if (m === 0 && k === 0) {
17              break;
18          }
19          let sum = 0;
20          console.log(sum, "\n");
21      }
22  })();

```

Go

```

1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var m, k int
9      for {
10         _, err := fmt.Scanf("%d %d", &m, &k)
11         if err != nil || (m == 0 && k == 0) {
12             break
13         }
14         sum := m + m/k
15         fmt.Println(sum)
16     }
17 }

```

11. 多组测试数据，首先输入一个整数N，接下来N行每行输入两个整数a和b，读取输入数据到Map

练习题 11. 共同祖先

C++

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main() {
5      int n, a, b;
6      vector<int> nums = vector<int>(30, 0); // 使用数组来记录映射关系，初始化为0
7      while (cin >> n) {
8          while (n-- > 0) {
9              cin >> a >> b;
10             nums[a] = b; // 记录映射关系
11         }
12     }
13 }

```

Java

```
1  import java.util.*;
2
3  public class Main{
4      static Map<Integer, Integer> map = new HashMap();
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          while (sc.hasNextInt()) {
8              int n = sc.nextInt();
9              for (int i = 0; i < n; i++) {
10                 int a = sc.nextInt();
11                 int b = sc.nextInt();
12                 map.put(a, b);
13             }
14         }
15     }
16 }
17 }
```

Python

```
1  while True:
2      try:
3          N = int(input())
4          myMap = {}
5          for _ in range(N):
6              a, b = map(int, input().split())
7              myMap[a] = b
8      except:
9          break
```

Js

```
1 // 引入readline模块来读取标准输入
2 const readline = require('readline')
3
4 // 创建readline接口
5 const rl = readline.createInterface({
6   input: process.stdin,
7   output: process.stdout
8 });
9
10 let myMap = new Array(21).fill(0)
11 let n
12 rl.on('line', (input) => {
13   // 读入每行数据, 将其转换为数组
14   const readline = input.split(' ').map(Number)
15   if (readline.length === 1) {
16     n = readline[0]
17   } else {
18     const [a, b] = readline
19     myMap[a] = b
20     n--
21   }
22 });
23
```

Go

```

1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      var n, a, b int
9      nums := make([]int, 30)
10
11     for {
12         _, err := fmt.Scanf("%d", &n)
13         if err != nil {
14             break
15         }
16
17         for i := 0; i < n; i++ {
18             fmt.Scanf("%d %d", &a, &b)
19             // 将输入数据放到map中
20             nums[a] = b
21         }
22     }
23 }

```

12. 多组测试数据。每组输入一个整数n，输出特定的数字图形

练习题 12. 打印数字图形

C++

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  void printTopPart(int n) {
6      for (int i = 1; i <= n; i++) {
7          // 打印空格
8          for (int j = 1; j <= n - i; ++j) {
9              cout << " ";
10         }
11         // 打印递增数字
12         for (int j = 1; j <= i; j++) {
13             cout << j;
14         }
15         // 打印递减数字
16         for (int j = i - 1; j >= 1; j--) {
17             cout << j;
18         }
19         cout << endl;
20     }
21 }
22
23 int main() {
24     int n;
25     while (cin >> n) {
26         if (n < 1 || n > 9) {
27             cout << "输入的整数n超出范围" << endl;
28         }
29         printTopPart(n);
30     }
31 }

```

Java

```

1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          while (scanner.hasNext()) {
7              int n = scanner.nextInt();
8              for (int i = 1; i <= n; i++) {
9                  print(n - i, i);
10             }
11             for (int i = n - 1; i >= 1; i--) {
12                 print(n - i, i);
13             }
14         }
15     }
16
17     public static void print(int blank, int n) {
18         // 前面需要补齐空格
19         for (int i = 0; i < blank; i++) {
20             System.out.print(" ");
21         }
22         for (int i = 1; i <= n; i++) {
23             System.out.print(i);
24         }
25         for (int i = n - 1; i > 0; i--) {
26             System.out.print(i);
27         }
28         System.out.println();
29     }
30 }

```

Python


```
1  while True:
2      try:
3          n = int(input())
4          for i in range(1, n + 1):
5              print(' ' * (n - i), end='')
6              print(''.join(map(str, range(1, i + 1))) + ''.join(map(str, ra
nge(i - 1, 0, -1))))
7          for i in range(n - 1, 0, -1):
8              print(' ' * (n - i), end='')
9              print(''.join(map(str, range(1, i + 1))) + ''.join(map(str, ra
nge(i - 1, 0, -1))))
10     except:
11         break
```

Go

```

1  package main
2
3  import "fmt"
4
5  func printTopPart(n int) {
6      for i := 1; i <= n; i++ {
7          // 打印空格
8          for j := 1; j <= n-i; j++ {
9              fmt.Print(" ")
10             }
11             // 打印递增数字
12             for j := 1; j <= i; j++ {
13                 fmt.Print(j)
14             }
15             // 打印递减数字
16             for j := i - 1; j >= 1; j-- {
17                 fmt.Print(j)
18             }
19             fmt.Println()
20         }
21     }
22
23     func main() {
24         var n int
25         for {
26             _, err := fmt.Scan(&n)
27             if err != nil {
28                 break
29             }
30
31             if n < 1 || n > 9 {
32                 fmt.Println("输入的整数n超出范围")
33                 continue
34             }
35             printTopPart(n)
36         }
37     }

```

JS

```

1  function printTopPart(n) {
2      for (let i = 1; i <= n; i++) {
3          // 打印空格
4          for (let j = 1; j <= n - i; ++j) {
5              process.stdout.write(" ");
6          }
7
8          // 打印递增数字
9          for (let j = 1; j <= i; j++) {
10             process.stdout.write(String(j));
11         }
12
13         // 打印递减数字
14         for (let j = i - 1; j >= 1; j--) {
15             process.stdout.write(String(j));
16         }
17         console.log();
18     }
19 }
20
21 function main() {
22     const readline = require('readline');
23     const rl = readline.createInterface({
24         input: process.stdin,
25         output: process.stdout
26     });
27
28     rl.on('line', (input) => {
29         let n = parseInt(input);
30         if (n < 1 || n > 9) {
31             console.log("输入的整数n超出范围");
32         }
33         printTopPart(n);
34     });
35 }
36
37 main();

```

13. 多行输入，每行输入为一个字符和一个整数，遇到特殊字符结束

练习题 13. 镂空三角形

C++

```
1  ▾ int main() {
2      char c;
3      int n;
4  ▾  while(cin >> c){
5          if(c == '@')
6              break;
7          cin >> n;
8          myprint(c, n);
9      }
10     return 0;
11 }
```

Java

```
1  import java.util.Scanner;
2
3  ▾ public class Main {
4  ▾  public static void main(String[] args) {
5      Scanner sc = new Scanner(System.in);
6
7  ▾  while (sc.hasNext()) {
8      String line = sc.nextLine();
9      if (line.equals("@"))
10         break;
11
12         String[] inputs = line.split(" ");
13         char ch = inputs[0].charAt(0);
14         int n = Integer.parseInt(inputs[1]);
15     }
16     sc.close();
17 }
18
19 }
20
```

Python

```

1  while True:
2      try:
3          line = input()
4          if line == '@':
5              break
6          ch, n = line.split()
7          n = int(n)
8      except:

```

Go

```

1  package main
2
3  import (
4      "fmt"
5  )
6
7
8  func main() {
9      var n int
10     var a rune
11     for {
12         _, err := fmt.Scanf("%c", &a)
13         if err != nil || a == '@' {
14             break
15         }
16
17         _, err = fmt.Scanf("%d", &n)
18         if err != nil {
19             break
20         }
21     }
22 }
23 }

```

Js

```

1  function main() {
2      const readline = require('readline');
3      const rl = readline.createInterface({
4          input: process.stdin,
5          output: process.stdout
6      });
7
8      rl.on('line', (input) => {
9          let [c, n] = input.split(' ');
10         if (c === '@') {
11             rl.close(); // 结束输入监听
12             return;
13         }
14         myprint(c, parseInt(n));
15     });
16 }
17
18 main();

```

14. 第一行是一个整数n，表示一共有n组测试数据，之后输入n行字符串

练习题 14. 句子缩写

C++

```

1  #include<iostream>
2  #include<string>
3  using namespace std;
4
5  int main() {
6      int n;
7      string result, s;
8      cin >> n;
9      getchar(); // 吸收一个回车, 因为输入n之后, 要输入一个回车
10 while (n--> {
11     getline(cin, s);
12     for (int i = 1; i < s.size() - 1; i++) {
13
14     }
15     cout << result << endl;
16 }
17 }
18

```

Java

```

1  import java.util.*;
2
3  public class Main{
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          while (sc.hasNextInt()) {
7              int n = sc.nextInt();
8              sc.nextLine();
9              for (int i = 0; i < n; i++) {
10                 String line = sc.nextLine().trim();
11
12                 StringBuilder sb = new StringBuilder();
13
14                 System.out.println(sb.toString());
15             }
16         }
17     }
18 }

```

```

1  import java.util.Scanner;
2
3  public class Main{
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          int n=in.nextInt();
7          in.nextLine();
8          for (int j = 0; j < n; j++) {
9              String s=in.nextLine();
10             StringBuilder sb=new StringBuilder();
11
12             System.out.println(sb.toString().toUpperCase());
13         }
14     }
15 }

```

Python

```

1  T = int(input())
2  for _ in range(T):
3      words = input().split()
4      for word in words:
5

```

Go


```

1  package main
2
3  import (
4      "bufio"
5      "fmt"
6      "os"
7      "strings"
8  )
9
10 func main() {
11     var n int
12     _, err := fmt.Scan(&n)
13     if err != nil {
14         return
15     }
16     scanner := bufio.NewScanner(os.Stdin)
17     for i := 0; i < n && scanner.Scan(); i++ {
18         input := scanner.Text()
19         words := strings.Fields(input)
20         for _, word := range words {
21
22         }
23     }
24 }

```

Js

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5  async function main(){
6      const n=parseInt(await readline());
7      for(let i=0;i<n;i++){
8          let line=await readline();
9          let words=line.split(/\s+/);
10         words.forEach(item=>{
11             process.stdout.write()
12         });
13         console.log();
14     }
15 }
16
17 main();

```

15. 第一行是一个整数n，然后是n组数据，每组数据2行，每行为一个字符串，为每组数据输出一个字符串，每组输出占一行

练习题 15.神秘字符

C++

```
1  #include<iostream>
2  #include<string>
3  using namespace std;
4  int main() {
5      int n;
6      cin >> n;
7      getchar(); // 吸收n后的一个回车
8      while (n--) {
9          string s, t;
10         cin >> s >> t;
11
12         string result = "";
13
14         cout << result << endl;
15     }
16 }
17
```

Java

```

1  import java.util.Scanner;
2
3  public class Main{
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          int n = in.nextInt();
7          for (int i = 0; i < n; i++) {
8              String a = in.next();
9              String b = in.next();
10             StringBuilder sb = new StringBuilder(a);
11             System.out.println(sb.toString());
12         }
13     }
14 }

```

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.util.StringTokenizer;
5
6  public class Main {
7      public static void main(String[] args) throws IOException {
8          BufferedReader reader = new BufferedReader(new InputStreamReader(S
9              ystem.in));
10         String str = null;
11         while((str = reader.readLine()) != null){
12             StringTokenizer tokenizer = new StringTokenizer(str);
13             int n = Integer.parseInt(tokenizer.nextToken());
14             for(int i = 0; i < n; i++){
15                 String a = reader.readLine();
16                 String b = reader.readLine();
17                 StringBuilder sb = new StringBuilder();
18
19                 System.out.println(sb.toString());
20             }
21         }
22     }
23 }

```

Python

```

1  n = int(input())
2  for _ in range(n):
3      line1 = input()
4      line2 = input()
5      mid = len(line1) // 2
6      result = line1[:mid] + line2 + line1[mid:]
7      print(result)

```

Go

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var n int
7      _, err := fmt.Scan(&n)
8      if err != nil {
9          return
10     }
11     for n > 0 {
12         var a, b string
13         _, _ = fmt.Scanln(&a)
14         _, _ = fmt.Scanln(&b)
15         fmt.Println(a[:len(a)/2] + b + a[len(a)/2:])
16         n--
17     }
18 }

```

JS

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5
6  async function main(){
7      const n=parseInt(await readline());
8      for(let i=0;i<n;i++){
9          let str1=await readline();
10         let str2=await readline();
11         console.log();
12     }
13 }
14
15 main();

```

16. 多组测试数据，第一行是一个整数n，接下来是n组字符串，输出字符串

练习题：16. 位置互换

C++

```

1  #include<iostream>
2  #include<string>
3  using namespace std;
4
5  int main() {
6      int n;
7      cin >> n;
8      string s;
9      while (n-->0) {
10         cin >> s;
11
12         cout << s << endl;
13     }
14 }

```

Java

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.util.StringTokenizer;
5
6  public class Main {
7      public static void main(String[] args) throws IOException {
8          BufferedReader reader = new BufferedReader(new InputStreamReader(S
system.in));
9          String str = null;
10         while((str = reader.readLine()) != null){
11             StringTokenizer tokenizer = new StringTokenizer(str);
12             int n = Integer.parseInt(tokenizer.nextToken());
13             for(int i = 0; i < n; i++){
14                 String s = reader.readLine();
15                 StringBuilder sb = new StringBuilder();
16
17                 System.out.println(sb.toString());
18             }
19         }
20     }
21 }

```

```

1  // 方法二：原地交换
2  import java.util.Scanner;
3
4  public class Main {
5
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8
9          int n = sc.nextInt();
10         sc.nextLine();
11         for (int i = 0; i < n; i++) {
12             String s1 = sc.nextLine();
13             int len = s1.length();
14             char[] chs = s1.toCharArray();
15
16             System.out.println(new String(chs));
17         }
18         sc.close();
19     }
20
21 }
22

```

Python

```
1 C = int(input())
2 for _ in range(C):
3     s = input()
4
5     print(result)
```

Go

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func main() {
8     var n int
9
10    _, err := fmt.Scanf("%d", &n)
11    if err != nil {
12        return
13    }
14    for i := 0; i < n; i++ {
15        var s string
16        _, err = fmt.Scanf("%s", &s)
17        if err != nil {
18            return
19        }
20
21        fmt.Println(s)
22    }
23 }
24
25
```

Js

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5  async function main(){
6      let n=parseInt(await readline());
7      for(let i=0;i<n;i++){
8          let str=await readline();
9          console.log();
10     }
11 }
12
13 main();

```

**17. 多组测试数据，每组测试数据的第一行为整数N
($1 \leq N \leq 100$)，当N=0时，输入结束，第二行为N个正整数，
以空格隔开，输出结果为字符串**

练习题：17. 出栈合法性

C++


```

1  #include<iostream>
2  #include<stack>
3  #include<vector>
4  using namespace std;
5  int main() {
6      int n;
7      int nums[105];
8      while(cin >> n) {
9          // 结束输入
10         if (n == 0) break;
11         for (int index = 0; index < n; index++) cin >> nums[index]; // 输入数组
12         stack<int> st;
13         int index = 0;
14
15         // 输出字符串
16         if (st.empty() && index == n) cout << "Yes" << endl;
17         else cout << "No" << endl;
18     }
19 }

```

Java

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.util.StringTokenizer;
5
6  public class Main {
7      public static void main(String[] args) throws IOException {
8          BufferedReader reader = new BufferedReader(new InputStreamReader(S
system.in));
9          String str = null;
10         while((str = reader.readLine()) != null){
11             StringTokenizer tokenizer = new StringTokenizer(str);
12             // 读取n
13             int n = Integer.parseInt(tokenizer.nextToken());
14             if(n == 0){
15                 break;
16             }
17             int[] arr = new int[n];
18             tokenizer = new StringTokenizer(reader.readLine());
19             // 读取n个正整数
20             for(int i = 0; i < n; i++){
21                 arr[i] = Integer.parseInt(tokenizer.nextToken());
22             }
23             if(check(arr)){
24                 System.out.println("Yes");
25             }else{
26                 System.out.println("No");
27             }
28         }
29     }
30 }

```

```

1 // 方法二：使用栈模拟
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         while (true) {
10             int n = sc.nextInt();
11             if (n == 0)
12                 break;
13
14             Stack<Integer> stack = new Stack<>();
15             for (int i = 0; i < n; i++)
16                 stack.push(sc.nextInt());
17             }
18             if (isValidPopSequence(n, poppedSequence))
19                 System.out.println("Yes");
20             else
21                 System.out.println("No");
22         }
23         sc.close();
24     }
25 }
26

```

Python

```

1 while True:
2     n = int(input())
3     if n == 0:
4         break
5     sequence = list(map(int, input().split()))
6     stack = []
7     possible = True
8     for num in sequence:
9         stack.append(num)
10    if possible:
11        print('Yes')
12    else:
13        print('No')

```

Go

```
1  package main
2
3  import (
4      "fmt"
5  )
6
7  func main() {
8      for {
9          var n int
10         _, _ = fmt.Scan(&n)
11
12         if n == 0 {
13             break
14         }
15
16         nums := make([]int, n)
17         for i := 0; i < n; i++ {
18             _, _ = fmt.Scan(&nums[i])
19         }
20
21         stack := make([]int, 0)
22
23         for i := 1; i <= n; i++ {
24             stack = append(stack, nums[i])
25         }
26     }
27 }
```

JS

```

1  const readline = require('readline')
2  const rl = readline.createInterface({
3      input: process.stdin,
4      output: process.stdout
5  })
6
7  // 因为只能一行一行地读取数据，所以用-1来表示是否是一组新的数据
8  let n = -1
9  rl.on('line', (input) => {
10     line = input.split(' ').map(Number)
11     if (line.length === 1 && n === -1) {
12         // 如果是一组新的数据，先读入n，直接return等待下一次输入
13         n = line[0]
14         return
15     }
16     // line保存的是出栈序列
17     if (islegal(line, n)) {
18         console.log('Yes')
19     } else {
20         console.log('No')
21     }
22     // 处理完一组数据后，写回n = -1
23     n = -1
24 })
25

```

18. 一组输入数据，第一行为n+1个整数，逆序插入n个整数，第二行为一个整数m，接下来有m行字符串，并根据字符串内容输入不同个数的数据

练习题：18. 链表的基本操作

C++

```

1  int main() {
2      int n, a, m, t, z;
3      string s;
4      cin >> n;
5
6      while (n--> 0) {
7          cin >> a;
8      }
9      cin >> m;
10     while (m--> 0) {
11         // 输入m个字符串，根据字符串内容输出
12         cin >> s;
13         if (s == "show") {
14             cout << "Link list is empty" << endl;
15         }
16         if (s == "delete") {
17             cin >> t;
18             // 本题的删除索引是从1开始，函数实现是从0开始，所以这里是 t - 1
19             if (deleteAtIndex(t - 1) == -1) cout << "delete fail" << endl;
20             else cout << "delete OK" << endl;
21         }
22         if (s == "insert") {
23             cin >> t >> z;
24             if (addAtIndex(t - 1, z) == -1) cout << "insert fail" << endl;
25             else cout << "insert OK" << endl;
26         }
27     }
28     if (s == "get") {
29         cin >> t;
30         int getValue = get(t - 1);
31         if (getValue == -1) cout << "get fail" << endl;
32         else cout << getValue << endl;
33     }
34 }
35 }

```

Java

```

1 public static void main(String[] args) {
2     Scanner sc = new Scanner(System.in);
3     // 输入n
4     int n = sc.nextInt();
5     // 输入n个整数
6     for (int i = 0; i < n; i++) {
7         int num = sc.nextInt();
8         linkedList.addFirst(num);
9     }
10    // 输入m
11    int m = sc.nextInt();
12
13    // 输入m个字符串
14    for (int i = 0; i < m; i++) {
15        // 获取输入的字符串
16        String operation = sc.next();
17        // 根据输入内容, 给出不同输出结果
18        if ("get".equals(operation)) {
19            int a = sc.nextInt();
20            int result = linkedList.get(a - 1);
21            if (result != -1) {
22                System.out.println(result);
23            } else {
24                System.out.println("get fail");
25            }
26        } else if ("delete".equals(operation)) {
27            int a = sc.nextInt();
28            boolean deleteResult = linkedList.delete(a - 1);
29            if (deleteResult) {
30                System.out.println("delete OK");
31            } else {
32                System.out.println("delete fail");
33            }
34        } else if ("insert".equals(operation)) {
35            int a = sc.nextInt();
36            int e = sc.nextInt();
37            boolean insertResult = linkedList.insert(a - 1, e);
38            if (insertResult) {
39                System.out.println("insert OK");
40            } else {
41                System.out.println("insert fail");
42            }
43        } else if ("show".equals(operation)) {
44            linkedList.show();
45        }
46    }
47    sc.close();

```

Python

```

1  if __name__ == "__main__":
2      while True:
3          mylinklist = MyLinkedList()
4          try:
5              # 读取链表长度和链表数值
6              n, *nums = list(map(int, input().split()))
7              # 初始化链表
8              for i in range(n):
9                  mylinklist.addAtHead(nums[i])
10             # 读取操作的个数
11             m = int(input())
12             for i in range(m):
13                 # 读取输入的操作和对应的索引
14                 s = input().split()
15                 if s[0] == "show":
16                     if mylinklist.printLinkedList() == -1:
17                         print("Link list is empty")
18                 if s[0] == "delete":
19                     t = int(s[1])
20                     if mylinklist.deleteAtIndex(t - 1) == -1:
21                         print("delete fail")
22                     else:
23                         print("delete OK")
24                 if s[0] == "insert":
25                     t = int(s[1])
26                     z = int(s[2])
27                     if mylinklist.addAtIndex(t - 1, z) == -1:
28                         print("insert fail")
29                     else:
30                         print("insert OK")
31                 if s[0] == "get":
32                     t = int(s[1])
33                     getValue = mylinklist.get(t - 1)
34                     if getValue == -1:
35                         print("get fail")
36                     else:
37                         print(getValue)
38             except:
39                 break

```


Go

```

1 func main() {
2     var n int
3     // 输入n
4     _, err := fmt.Scan(&n)
5     if err != nil {
6         return
7     }
8     list := Constructor()
9
10    // 输入n个数据
11    for i := 0; i < n; i++ {
12        var data int
13        _, err = fmt.Scan(&data)
14        if err != nil {
15            return
16        }
17        list.insert(1, data)
18    }
19    var m int
20    //输入m
21    _, err = fmt.Scan(&m)
22    if err != nil {
23        return
24    }
25    // 输入m行字符串
26    for i := 0; i < m; i++ {
27        var s string
28        _, err = fmt.Scan(&s)
29        if err != nil {
30            return
31        }
32        // 根据字符串操作输出
33        switch s {
34            case "get":
35                var index int
36                _, err = fmt.Scan(&index)
37                if err != nil {
38                    return
39                }
40                val, err := list.get(index)
41                if err != nil {
42                    fmt.Println(err.Error())
43                } else {
44                    fmt.Println(val)
45                }
46            case "delete":
47                var index int

```

```

48         _, err = fmt.Scan(&index)
49         if err != nil {
50             return
51         }
52         err := list.delete(index)
53         if err != nil {
54             fmt.Println(err.Error())
55         } else {
56             fmt.Println("delete OK")
57         }
58     case "insert":
59         var index, val int
60         _, err = fmt.Scan(&index, &val)
61         if err != nil {
62             return
63         }
64         if err = list.insert(index, val); err != nil {
65             fmt.Println(err.Error())
66         } else {
67             fmt.Println("insert OK")
68         }
69     case "show":
70         list.Show()
71     }
72 }
73 }
74

```

Javascript

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4  const out=process.stdout;
5
6  async function main(){
7      const nums=(await readline()).split(" ").map(Number);
8      let root=new Node(-1);
9      for(let i=1;i<=nums[0];i++){
10         root.insert(1,nums[i]);
11     }
12     const n=parseInt(await readline());
13     let index;
14     for(let i=0;i<n;i++){
15         let line=(await readline()).split(" ");
16         let op=line[0];
17         let flag=false;
18         switch(op){
19             case "show":
20                 root.show();
21                 break;
22             case "get":
23                 index=parseInt(line[1]);
24                 let node=root.getNode(index);
25                 if(node) out.write(node.data.toString());
26                 else out.write("get fail");
27                 break;
28             case "delete":
29                 index=parseInt(line[1]);
30                 flag=root.deleteNode(index);
31                 if(flag) out.write("delete OK");
32                 else out.write("delete fail");
33                 break;
34             case "insert":
35                 index=parseInt(line[1]);
36                 flag=root.insert(index,parseInt(line[2]));
37                 if(flag) out.write("insert OK");
38                 else out.write("insert fail");
39                 break;
40         }
41         console.log();
42     }
43 }
44
45 main();

```

19. 多组测试数据，每行为n+1个数字， 输出链表或对应的字符串

练习题：19. 单链表反转

练习题：20. 删除重复元素

C++

```
1  int main() {
2
3      int n, m;
4      ListNode* dummyHead = new ListNode(0); // 这里定义的头结点 是一个虚拟头结点，而不是真正的链表头结点
5      while (cin >> n) {
6          if (n == 0) {
7              cout << "list is empty" << endl;
8              continue;
9          }
10         ListNode* cur = dummyHead;
11         // 读取输入构建链表
12         while (n-- > 0) {
13             cin >> m;
14             ListNode* newNode = new ListNode(m); // 开始构造节点
15             cur->next = newNode;
16             cur = cur->next;
17         }
18         printLinkedList(dummyHead->next);
19         printLinkedList(reverseList(dummyHead->next));
20     }
21 }
22 // 输出链表
23 void printLinkedList(ListNode* head) {
24     ListNode* cur = head;
25     while (cur != nullptr) {
26         cout << cur->val << " ";
27         cur = cur->next;
28     }
29     cout << endl;
30 }
```

Java

```

1  import java.util.Scanner;
2
3  public class Main{
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          while (sc.hasNextLine()) {
7              String[] str = sc.nextLine().split(" ");
8              if (Integer.parseInt(str[0]) == 0) {
9                  System.out.println("list is empty");
10             }
11             ListNode dummyhead = new ListNode(-1);
12             ListNode cur = dummyhead;
13             //构造链表
14             for (int i = 1; i < str.length; i++) {
15                 ListNode temp = new ListNode(Integer.parseInt(str[i]));
16                 cur.next = temp;
17                 cur = cur.next;
18                 if (i == str.length - 1) cur.next = null;
19             }
20             //输出原链表
21             ListNode pointer = dummyhead.next;
22             while (pointer != null) {
23                 System.out.print(pointer.val + " ");
24                 pointer = pointer.next;
25             }
26             System.out.println();
27         }
28     }
29 }

```

python

```

1  # 打印链表
2  def printLinkedList(head: ListNode):
3      cur = head
4      while cur:
5          print(cur.val, end = " ")
6          cur = cur.next
7      print()
8  if __name__ == "__main__":
9      while True:
10         try:
11             # 输入5 1 2 3 4 5, 表示链表有5个节点, 值分别为1 2 3 4 5
12             n, *nums = map(int, input().split())
13         except:
14             break
15         if n == 0:
16             print("list is empty")
17             continue
18         dummyHead = ListNode(0) # 这里定义的头结点 是一个虚拟头结点, 而不是真正
    的链表头结点
19         cur = dummyHead
20         for i in range(n): # 开始构造节点
21             cur.next = ListNode(nums[i])
22             cur = cur.next
23         printLinkedList(dummyHead.next) # 打印链表
24         printLinkedList(reverseList(dummyHead.next)) # 打印翻转后的链表

```

Go

```

1 func main() {
2     for {
3         var n int
4         _, err := fmt.Scan(&n)
5         if err != nil {
6             return
7         }
8         if n == 0 {
9             fmt.Println("list is empty")
10            continue
11        }
12        // 构建链表
13        dummyHead := &Node{}
14        cur := dummyHead
15        for n > 0 {
16            var val int
17            _, err = fmt.Scan(&val)
18            if err != nil {
19                return
20            }
21            node := &Node{val: val}
22            cur.next = node
23            cur = cur.next
24            n--
25        }
26        show(dummyHead.next)
27    }
28 }
29 // 输出链表
30 func show(head *Node) {
31     if head == nil {
32         return
33     }
34     cur := head
35     for {
36         fmt.Printf("%d ", cur.val)
37         cur = cur.next
38         if cur == nil {
39             fmt.Println()
40             return
41         }
42     }
43 }

```



```

1  // 引入readline模块来读取标准输入
2  const readline = require('readline')
3
4  // 创建readline接口
5  const rl = readline.createInterface({
6      input: process.stdin,
7      output: process.stdout
8  })
9
10 // 处理输入和输出
11 rl.on('line', (input) => {
12     // 将每一行以空格分割成一个字符串数组，并将每个元素转换成number类型
13     const line = input.split(' ').filter(item => item !== '').map(Number)
14
15     // 第一个元素是链表长度
16     const n = line[0]
17
18     // 长度为0，直接输出 list is empty
19     if (n === 0) {
20         console.log('list is empty')
21         return
22     }
23     // 根据给定输入创建链表
24     let head = createLinkedList(line.slice(1))
25     // 打印翻转前的链表
26     printLinkedList(head)
27 })
28
29
30 // 给定一个number数组，创建出链表，返回链表的头节点
31 function createLinkedList(arr) {
32     // 创建头节点
33     const head = new Node(arr[0])
34     // 初始化尾指针，方便添加新的节点
35     let tail = head
36     arr.slice(1).forEach(item => {
37         // 每次将细节点插在尾节点后面
38         tail.next = new Node(item)
39         // 更新尾节点为新创建的节点
40         tail = tail.next
41     })
42     // 返回头节点
43     return head
44 }
45
46 // 输出链表
47 function printLinkedList(head) {

```

```
48     let output = ''
49     // 将每个节点的val拼接成一个字符串
50     while(head) {
51         output += `${head.val} `
52         head = head.next
53     }
54     // 最后输出
55     console.log(output)
56 }
```

20. 多组输入，每组输入包含两个字符串，输出字符串

练习题：21. 构造二叉树

C++

```

1  int main() {
2
3      string s;
4      while (getline(cin, s)) { // 接受一整行字符串
5          string preorder = "", inorder = "";
6          // 拆分出两个字符串
7          int i;
8          for (i = 0; s[i] != ' '; i++) preorder += s[i];
9          i++;
10         for (; i < s.size(); i++) inorder += s[i];
11
12         // 开始构造二叉树
13         TreeNode* root = buildTree(preorder, inorder);
14         // 输出后序遍历结果
15         postorderTraversal(root);
16         cout << endl;
17     }
18     return 0;
19 }
20
21 // 后序遍历二叉树
22 void postorderTraversal(TreeNode* root) {
23     if (root == nullptr) {
24         return;
25     }
26
27     postorderTraversal(root->left);
28     postorderTraversal(root->right);
29     cout << root->val;
30 }

```

Java

```

1  public class Main{
2      public static Map<Character, Integer> map = new HashMap();
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          while (sc.hasNextLine()) {
6              String s = sc.nextLine();
7              String[] ss = s.split(" ");
8              String pre = ss[0];
9              String in = ss[1];
10             // 构建二叉树
11             TreeNode res = afterHelper(pre.toCharArray(), in.toCharArray()
12         );
13             //打印二叉树
14             printTree(res);
15             System.out.println();
16         }
17     }
18     public static void printTree(TreeNode root) {
19         if (root == null) return;
20         printTree(root.left);
21         printTree(root.right);
22         System.out.print(root.val);
23     }
24 }

```

Python

```

1  def postorder_traversal(root):
2      if not root:
3          return []
4      left = postorder_traversal(root.left)
5      right = postorder_traversal(root.right)
6      return left + right + [root.val]
7
8  while True:
9      try:
10         preorder, inorder = map(str, input().split())
11         if not preorder or not inorder:
12             break
13         root = build_tree(preorder, inorder)
14         postorder = postorder_traversal(root)
15         print(''.join(postorder))
16     except EOFError:
17         break

```

Go

```

1  package main
2
3  import (
4      "fmt"
5      "strings"
6  )
7
8  func main() {
9      for {
10         var preorder, inorder string
11         _, err := fmt.Scan(&preorder, &inorder)
12         if err != nil {
13             return
14         }
15         tree := buildTree(preorder, inorder, 0, len(preorder)-1, 0, len(in
order)-1)
16         fmt.Println(postorderTraversal(tree))
17     }
18 }
19 // 后序遍历结果
20 func postorderTraversal(root *TreeNode) string {
21     var res []string
22     var traversal func(node *TreeNode)
23     traversal = func(node *TreeNode) {
24         if node == nil {
25             return
26         }
27         traversal(node.Left)
28         traversal(node.Right)
29         res = append(res, node.Val)
30     }
31     traversal(root)
32     return strings.Join(res, "")
33 }

```

JS

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5  const out=process.stdout;
6
7  class Node{
8      constructor(data){
9          this.data=data;
10         this.left=null;
11         this.right=null;
12     }
13     postOrder(){
14         if(this.left!=null) this.left.postOrder();
15         if(this.right!=null) this.right.postOrder();
16         out.write(this.data);
17     }
18 }
19
20 async function main(){
21     while(line=await readline()){
22         [preOrder,inOrder]=line.split(" ");
23         const root=createBTree(preOrder,inOrder,0,preOrder.length-1,0,inOrder.
length-1);
24         // 获取后序遍历
25         root.postOrder();
26         console.log();
27     }
28 }

```

21. 一组多行数据，第一行为数字n，表示后面有n行，后面每行为1个字符加2个整数，输出树节点的后序遍历字符串

练习题：22. 二叉树的遍历

C++

```

1  #include <iostream>
2  #include <unordered_map>
3  #include <vector>
4  using namespace std;
5
6
7  // 前序遍历二叉树
8  void preorderTraversal(TreeNode* root) {
9      if (root == nullptr) {
10         return;
11     }
12
13     cout << root->val;
14     preorderTraversal(root->left);
15     preorderTraversal(root->right);
16 }
17
18 // 中序遍历二叉树
19 void inorderTraversal(TreeNode* root) {
20     if (root == nullptr) {
21         return;
22     }
23
24     inorderTraversal(root->left);
25     cout << root->val;
26     inorderTraversal(root->right);
27 }
28
29 // 后序遍历二叉树
30 void postorderTraversal(TreeNode* root) {
31     if (root == nullptr) {
32         return;
33     }
34
35     postorderTraversal(root->left);
36     postorderTraversal(root->right);
37     cout << root->val;
38 }
39
40 int main() {
41     int n;
42     cin >> n;
43     unordered_map<char, std::pair<char, char>> nodeMap;
44
45     // 先保存输入的数据
46     vector<char> index = vector<char>(n + 1, '0');
47

```



```

48     vector<vector<int>> nums = vector<vector<int>>(n + 1, vector<int>(2, 0
49 ));
50     for (int i = 1; i <= n; i++) {
51         cin >> index[i] >> nums[i][0] >> nums[i][1];
52     }
53
54     // 输出
55     preorderTraversal(root);
56     cout << std::endl;
57
58     inorderTraversal(root);
59     cout << std::endl;
60
61     postorderTraversal(root);
62     cout << std::endl;
63
64     return 0;

```

Java

```

1  import java.util.*;
2
3  class TreeNode {
4      char val;
5      TreeNode left;
6      TreeNode right;
7      public TreeNode(char val) {
8          this.val = val;
9      }
10 }
11 public class Main{
12     static TreeNode[] nodes = new TreeNode[30];
13
14     public static void main(String[] args) {
15         Scanner sc = new Scanner(System.in);
16         while (sc.hasNextInt()) {
17             int len = sc.nextInt();
18             for (int i = 0; i < len; i++) {
19                 // 获取字符和左右子节点
20                 char val = sc.next().charAt(0);
21                 int left = sc.nextInt();
22                 int right = sc.nextInt();
23
24             }
25             preorder(nodes[1]);
26             System.out.println();
27             inorder(nodes[1]);
28             System.out.println();
29             postorder(nodes[1]);
30             System.out.println();
31         }
32     }
33     public static void preorder(TreeNode root) {
34         if (root == null) return;
35         System.out.print(root.val);
36         preorder(root.left);
37         preorder(root.right);
38     }
39     public static void inorder(TreeNode root) {
40         if (root == null) return;
41         inorder(root.left);
42         System.out.print(root.val);
43         inorder(root.right);
44     }
45     public static void postorder(TreeNode root) {
46         if (root == null) return;
47         postorder(root.left);

```

```
48         postorder(root.right);
49         System.out.print(root.val);
50     }
51 }
```

```

1 // 方法二：使用索引，简化构建树的过程
2 import java.util.Scanner;
3
4 public class Main {
5     static class TreeNode {
6         char val;
7         int left;
8         int right;
9
10        public TreeNode(char val, int left, int right) {
11            this.val = val;
12            this.left = left;
13            this.right = right;
14        }
15    }
16
17    static TreeNode[] nodes;
18
19    public static void main(String[] args) {
20        Scanner sc = new Scanner(System.in);
21        int n = sc.nextInt();
22        nodes = new TreeNode[n + 1];
23        for (int i = 0; i < n; i++) {
24            char val = sc.next().charAt(0);
25            int left = sc.nextInt();
26            int right = sc.nextInt();
27            nodes[i + 1] = new TreeNode(val, left, right);
28        }
29        preOrderTraversal(1);
30        System.out.println();
31        inOrderTraversal(1);
32        System.out.println();
33        postOrderTraversal(1);
34        System.out.println();
35        sc.close();
36    }
37
38    private static void postOrderTraversal(int root) {
39        if (root == 0)
40            return;
41        postOrderTraversal(nodes[root].left);
42        postOrderTraversal(nodes[root].right);
43        System.out.print(nodes[root].val);
44    }
45
46    private static void inOrderTraversal(int root) {
47        if (root == 0)

```

```
48         return;
49         inOrderTraversal(nodes[root].left);
50         System.out.print(nodes[root].val);
51         inOrderTraversal(nodes[root].right);
52     }
53
54     private static void preOrderTraversal(int root) {
55         if (root == 0)
56             return;
57         System.out.print(nodes[root].val);
58         preOrderTraversal(nodes[root].left);
59         preOrderTraversal(nodes[root].right);
60     }
61
62 }
```

Go

```

1  package main
2
3  import (
4      "fmt"
5      "strings"
6  )
7
8  func preorder(root *Node) []string {
9      if root == nil {
10         return []string{}
11     }
12     left := preorder(root.Left)
13     right := preorder(root.Right)
14     return append([]string{root.Val}, append(left, right...)...)
15 }
16
17 func inorder(root *Node) []string {
18     if root == nil {
19         return []string{}
20     }
21     left := inorder(root.Left)
22     right := inorder(root.Right)
23     return append(append(left, root.Val), right...)
24 }
25
26 func postorder(root *Node) []string {
27     if root == nil {
28         return []string{}
29     }
30     left := postorder(root.Left)
31     right := postorder(root.Right)
32     return append(append(left, right...), root.Val)
33 }
34
35 func main() {
36     var n int
37     fmt.Scan(&n)
38
39     nodes := make([]*Node, n+1)
40     var line string
41     for i := 0; i < n; i++ {
42         fmt.Scan(&line)
43         val := line[0:1]
44         left, right := 0, 0
45         fmt.Scan(&left, &right)
46     }
47

```

```
48     root := nodes[1]
49     pre := preorder(root)
50     ino := inorder(root)
51     post := postorder(root)
52
53     fmt.Println(strings.Join(pre, ""))
54     fmt.Println(strings.Join(ino, ""))
55     fmt.Println(strings.Join(post, ""))
56 }
```

Python

```

1  def preorder(root):
2      if not root:
3          return []
4      left = preorder(root.left)
5      right = preorder(root.right)
6      return [root.val] + left + right
7
8  def inorder(root):
9      if not root:
10         return []
11     left = inorder(root.left)
12     right = inorder(root.right)
13     return left + [root.val] + right
14
15  def postorder(root):
16      if not root:
17         return []
18     left = postorder(root.left)
19     right = postorder(root.right)
20     return left + right + [root.val]
21
22  n = int(input())
23  nodes = [None] * (n + 1)
24  line_in = []
25  for i in range(n):
26      line = input().split()
27      val, left, right = line[0], int(line[1]), int(line[2])
28  root = nodes[1]
29  pre = preorder(root)
30  ino = inorder(root)
31  post = postorder(root)
32  print(''.join(pre))
33  print(''.join(ino))
34  print(''.join(post))

```

JS


```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5  const out=process.stdout;
6
7  class Node{
8      nodes=new Array();
9      constructor(data,left,right){
10         this.data=data;
11         this.left=left;
12         this.right=right;
13     }
14     preOrder(){
15         out.write(this.data);
16         if(this.left!==0) Node.nodes[this.left].preOrder();
17         if(this.right!==0) Node.nodes[this.right].preOrder();
18     }
19     inOrder(){
20         if(this.left!==0) Node.nodes[this.left].inOrder();
21         out.write(this.data);
22         if(this.right!==0) Node.nodes[this.right].inOrder();
23     }
24     postOrder(){
25         if(this.left!==0) Node.nodes[this.left].postOrder();
26         if(this.right!==0) Node.nodes[this.right].postOrder();
27         out.write(this.data);
28     }
29 }
30
31 async function main(){
32     const n=parseInt(await readline());
33     Node.nodes=new Array(n+1);
34     for(let i=1;i<=n;i++){
35         let line=(await readline()).split(" ");
36         let left=parseInt(line[1]);
37         let right=parseInt(line[2]);
38         Node.nodes[i]=new Node(line[0],left,right);
39     }
40     Node.nodes[1].preOrder();
41     console.log();
42     Node.nodes[1].inOrder();
43     console.log();
44     Node.nodes[1].postOrder();
45 }
46
47 main();

```

22. 多组测试数据，首先给出正整数N，接着输入两行字符串，字符串长度为N

练习题：23. 二叉树的高度

C++

```

1  #include <iostream>
2  #include <string>
3  #include <unordered_map>
4  using namespace std;
5
6  // 计算二叉树的高度
7  int getHeight(TreeNode* root) {
8      if (root == nullptr) {
9          return 0;
10     }
11
12     int leftHeight = getHeight(root->left);
13     int rightHeight = getHeight(root->right);
14
15     return max(leftHeight, rightHeight) + 1;
16 }
17
18 int main() {
19     int n;
20     while (cin >> n) {
21         string preorder, inorder;
22         cin >> preorder >> inorder;
23
24         unordered_map<char, int> indexMap;
25         for (int i = 0; i < n; ++i) {
26             indexMap[inorder[i]] = i;
27         }
28         TreeNode* root = buildTree(preorder, inorder, 0, 0, n - 1, indexMa
29 p);
30         int height = getHeight(root);
31         cout << height << endl;
32     }
33     return 0;
34 }

```

Java

```

1 // 方法一：递归
2 import java.util.Scanner;
3
4 public class Main {
5
6     static class TreeNode {
7         char val;
8         TreeNode left;
9         TreeNode right;
10
11     }
12     TreeNode(char val) {
13         this.val = val;
14         this.left = null;
15         this.right = null;
16     }
17
18     private static int getHeight(TreeNode root) {
19         if (root == null)
20             return 0;
21
22         int leftHeight = getHeight(root.left);
23         int rightHeight = getHeight(root.right);
24
25         return Math.max(leftHeight, rightHeight) + 1;
26     }
27
28     public static void main(String[] args) {
29         Scanner sc = new Scanner(System.in);
30
31         while (sc.hasNext()) {
32             sc.nextInt();
33             String preOrder = sc.next();
34             String inOrder = sc.next();
35
36             TreeNode root = buildTree(preOrder, inOrder);
37             int height = getHeight(root);
38             System.out.println(height);
39         }
40
41         sc.close();
42     }
43 }
44

```

```

1  // 方法二：递归（使用哈希表来优化中序遍历中查找根节点位置的过程）
2  import java.util.HashMap;
3  import java.util.Scanner;
4
5  public class Main {
6
7      static class TreeNode {
8          char val;
9          TreeNode left;
10         TreeNode right;
11
12         TreeNode(char val) {
13             this.val = val;
14             this.left = null;
15             this.right = null;
16         }
17     }
18     public static void main(String[] args) {
19         Scanner sc = new Scanner(System.in);
20
21         while (sc.hasNext()) {
22             int N = sc.nextInt();
23             String preOrder = sc.next();
24             String inOrder = sc.next();
25
26             HashMap<Character, Integer> inOrderMap = new HashMap<>();
27             for (int i = 0; i < N; i++) {
28                 inOrderMap.put(inOrder.charAt(i), i);
29             }
30
31             TreeNode root = buildTree(preOrder, 0, N - 1, 0, N - 1, inOrderMap);
32
33             int height = getHeight(root);
34             System.out.println(height);
35         }
36         sc.close();
37     }
38
39     private static int getHeight(TreeNode root) {
40         if (root == null) {
41             return 0;
42         }
43
44         int leftHeight = getHeight(root.left);
45         int rightHeight = getHeight(root.right);
46

```

```

47         return Math.max(leftHeight, rightHeight) + 1;
48     }
49 }
50

```

Python

```

1  class TreeNode:
2      def __init__(self, val):
3          self.val = val
4          self.left = None
5          self.right = None
6
7  def get_height(root):
8      if not root:
9          return 0
10
11     left_height = get_height(root.left)
12     right_height = get_height(root.right)
13
14     return max(left_height, right_height) + 1
15
16 def main():
17     while True:
18         try:
19             N = int(input())
20             pre_order = input().strip()
21             in_order = input().strip()
22
23             in_order_map = {}
24             for i in range(N):
25                 in_order_map[in_order[i]] = i
26             root = build_tree(pre_order, 0, N - 1, 0, N - 1, in_order_map)
27             height = get_height(root)
28             print(height)
29         except EOFError:
30             break
31
32 if __name__ == "__main__":
33     main()

```

Go

```

1  package main
2
3  import "fmt"
4
5  // 定义二叉树结构体
6  type treeNode struct {
7      val byte // 节点的值
8      left *treeNode // 左子树
9      right *treeNode // 右子树
10 }
11
12 // 计算二叉树的高度（深度）
13 func height(root *treeNode) int {
14     if root == nil {
15         return 0
16     }
17
18     leftHeight := height(root.left)
19     rightHeight := height(root.right)
20
21     if leftHeight > rightHeight {
22         return leftHeight + 1
23     } else {
24         return rightHeight + 1
25     }
26 }
27
28 func main() {
29     var k int
30     for {
31         _, err := fmt.Scan(&k)
32         if err != nil {
33             break
34         }
35         preorder := make([]byte, k)
36         inorder := make([]byte, k)
37
38         fmt.Scan(&preorder, &inorder)
39
40         // 构建二叉树
41         root := buildTree(preorder, inorder)
42
43         // 计算二叉树高度
44         fmt.Println(height(root))
45     }
46 }

```

```

1  const rl=require("readline").createInterface({input:process.stdin});
2  const iter=rl[Symbol.asyncIterator]();
3  const readline=async ()=>(await iter.next()).value;
4
5  const out=process.stdout;
6
7  class Node{
8    constructor(data){
9      this.data=data;
10     this.left=null;
11     this.right=null;
12   }
13   getHeight(){
14     let leftHeight=0,rightHeight=0;
15     if(this.left!=null) leftHeight=this.left.getHeight();
16     if(this.right!=null) rightHeight=this.right.getHeight();
17     return Math.max(leftHeight,rightHeight)+1;
18   }
19 }
20
21 async function main(){
22   while(line=await readline()){
23     // 获取输入的n
24     n=parseInt(line);
25     // 获取第一行字符串
26     let preOrder=await readline();
27     // 获取第二行字符串
28     let inOrder=await readline();
29     let root=Node.createTree(preOrder,inOrder,0,n-1,0,n-1);
30     console.log(root.getHeight());
31   }
32 }
33
34 main();

```

23. 多组测试数据。每组输入占一行，为两个字符串，由若干个空格分隔

练习题：24.最长公共子序列


```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5  int main() {
6      string text1, text2;
7      while (cin >> text1 >> text2) {
8          // 初始化dp数组
9          vector<vector<int>> dp(text1.size() + 1, vector<int>(text2.size()
+ 1, 0));
10
11         // 输出结果
12         cout << dp[text1.size()][text2.size()] << endl;
13     }
14     return 0;
15 }

```

Java

```

1  import java.util.Scanner;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner scanner = new Scanner(System.in);
5          while (scanner.hasNextLine()) {
6              String line = scanner.nextLine();
7              String[] s = line.split(" ");
8              String x = s[0];
9              String y = s[1];
10             int m = x.length();
11             int n = y.length();
12             // 初始化dp数组
13             int[][] dp = new int[m + 1][n + 1];
14             // 输出
15             int max = dp[m][n];
16             System.out.println(max);
17         }
18     }
19 }
20 }

```

Go

```

1  package main
2
3  import (
4      "fmt"
5  )
6
7
8  func longestCommonSubsequence(X, Y string) int {
9      m := len(X)
10     n := len(Y)
11     // 创建一个二维数组dp
12     dp := make([][]int, m+1)
13     for i := 0; i <= m; i++ {
14         dp[i] = make([]int, n+1)
15     }
16
17     return dp[m][n]
18 }
19
20 func main() {
21     var X, Y string
22     for {
23         // 输入两个字符串
24         _, err := fmt.Scan(&X, &Y)
25         if err != nil {
26             break
27         }
28         result := longestCommonSubsequence(X, Y)
29         fmt.Println(result)
30     }
31 }

```

Python

```

1  while True:
2      try:
3          text1, text2 = input().split()
4      except:
5          break
6
7      dp = [[0] * (len(text2) + 1) for _ in range(len(text1) + 1)]
8
9      print(dp[len(text1)][len(text2)])

```

Js

```
1  const readline = require('readline');
2
3  const rl = readline.createInterface({
4    input: process.stdin,
5    output: process.stdout,
6  })
7
8  rl.on('line', function(line) {
9    const input = line.split(' ');
10    const str1 = input[0], str2 = input[1];
11    const len1 = str1.length, len2 = str2.length;
12    // 初始化dp数组
13    const dp = new Array(len1 + 1).fill(0).map(() => new Array(len2 + 1).fill(0));
14
15    // 输出
16    console.log(dp[len1][len2]);
17  })
```

24. 多组测试数据，每组第一行为两个正整数n和m，接下来m行，每行3个整数，最后一行两个整数

练习题：25. 最爱的城市

C++

```

1  ▾ int main() {
2      int n, m;
3  ▾  while (cin >> n >> m) {
4      // 构建图
5  ▾  while (m--) {
6          int a, b, l;
7          std::cin >> a >> b >> l;
8      }
9
10     int x, y;
11     std::cin >> x >> y;
12
13 }
14 return 0;
15 }

```

```

1  ▾ int main() {
2      int n, m;
3  ▾  while (cin >> n >> m) {
4      // 构建图
5  ▾  for (int i = 0; i < m; i++) {
6          int a, b, l;
7          cin >> a >> b >> l;
8      }
9
10     int x, y;
11     std::cin >> x >> y;
12
13 }
14 return 0;
15 }

```

Java

```

1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class Main {
5
6      public static void main(String[] args) {
7          Scanner scanner = new Scanner(System.in);
8          while (scanner.hasNext()) {
9              // 处理输入
10             int n = scanner.nextInt();
11             int m = scanner.nextInt();
12
13             for (int i = 0; i < m; i++) {
14                 int a = scanner.nextInt();
15                 int b = scanner.nextInt();
16                 int l = scanner.nextInt();
17             }
18             int x = scanner.nextInt();
19             int y = scanner.nextInt();
20
21             // 处理输出
22             int res = dfs(graph, x, y, isVisit, sum);
23             if (res != Integer.MAX_VALUE) {
24                 System.out.println(res);
25             } else {
26                 System.out.println("No path");
27             }
28         }
29     }
30
31     private static int dfs(int[][] graph, int start, int end, int[] isVisit, int sum) {
32         if (end == start) {
33             return sum;
34         }
35         return min;
36     }
37 }

```

Python

```

1  def main():
2      while True:
3          try:
4              # 接收一行作为输入，将之分隔成n, m
5              n, m = map(int, input().split())
6              # 接收m行作为输入
7              for i in range(m):
8                  a, b, l = map(int, input().split())
9                  # 接收一行作为输入，将之分隔成x, y
10                 x, y = map(int, input().split())
11                 if dist[x][y] == float('inf'):
12                     print("No path")
13                 else:
14                     print(dist[x][y])
15
16             except EOFError:
17                 break
18
19
20 if __name__ == "__main__":
21     main()

```

Go

```

1 func main() {
2     for {
3         // 接收n和m
4         var n, m int
5         if _, err := fmt.Scan(&n, &m); err != nil {
6             break
7         }
8         // 接收m行数据
9         for i := 0; i < m; i++ {
10             var a, b, l int
11             fmt.Scan(&a, &b, &l)
12         }
13
14         // 接收最后一行两个数据
15         var x, y int
16         fmt.Scan(&x, &y)
17
18         if graph[x][y] != math.MaxInt32 {
19             fmt.Println(graph[x][y])
20         } else {
21             fmt.Println("No path")
22         }
23     }
24 }

```

JS

```

1  const readline = require('readline')
2  const rl = readline.createInterface({
3      input: process.stdin,
4      output: process.stdout
5  })
6  let m, n
7  let input = []
8  rl.on('line', (line) => {
9      input.push(line)
10 })
11   .on('close', () => {
12     let index = 0
13     while(index < input.length) {
14       const [n,m] = input[index++].split(' ').map(Number)
15       const edges = []
16       for(let i = 0; i < m; i++) {
17         const [a,b,l] = input[index++].split(' ').map(Number)
18         edges.push([a,b,l])
19       }
20       const [x,y] = input[index++].split(' ').map(Number)
21       const result = floydWarshall(n,m,edges,x,y)
22       if(result === INF) {
23         console.log('No path')
24       } else {
25         console.log(result)
26       }
27     }
28   })

```